

vasic-digital

Vasic Digital

Héroe

Ingeniería nativa de AI, construida para ser confiable.

Cualquiera puede conectar una aplicación a un LLM en una tarde. Lo difícil —lo que determina si un sistema de AI es solo una demo o un producto confiable— es todo lo que rodea al modelo: la abstracción del proveedor que sobrevive a una caída, la orquestación que mantiene a los agentes enfocados en su tarea, la verificación que detecta cuando un modelo intenta engañar, y la gobernanza que demuestra que todo el sistema se comporta como debe. Ese es el trabajo duro que realiza Vasic Digital. Diseñamos y entregamos sistemas de desarrollo de AI —los modelos, agentes, orquestación e infraestructura que convierten los grandes modelos de lenguaje en software confiable— junto con la capa de gobernanza que los mantiene honestos. Todo ello se rige por una regla inquebrantable: una funcionalidad no está “terminada” cuando pasan las pruebas; está terminada cuando un usuario real puede usarla, y existe evidencia registrada que lo demuestra.

Acerca de

Vasic Digital es una práctica de ingeniería enfocada en construir una familia interconectada de productos y módulos reutilizables para el desarrollo de AI. En lugar de un monolito, el trabajo se organiza como una flota: grandes aplicaciones de producto sobre docenas de módulos pequeños, probados de forma independiente y desacoplados, de modo que las partes validadas se reutilizan en todos los productos en lugar de reconstruirse. El lenguaje base es **Go**, complementado por **Kotlin / Kotlin Multiplatform**, **TypeScript/React**, **Python**, **Swift** y **Shell**, seleccionados según la tarea: Go para servicios y bibliotecas de alto rendimiento, Kotlin para herramientas de aprovisionamiento y desarrollo móvil multiplataforma, TypeScript para interfaces tipadas, Python para la integración de AI/ML.

Lo que une a la flota es la disciplina convertida en mecánica, no en aspiración. Cada proyecto hereda un **Constitution** de ingeniería compartido como submódulo de Git —de modo que una regla ajustada una vez se propaga en una flota de más de 140 repositorios— y toda capacidad que anuncia un producto debe estar respaldada por una prueba automatizada que genere evidencia antes de considerarse entregada. Esto no es lenguaje de marketing superpuesto al trabajo; es el modelo operativo sobre el que funciona. El efecto acumulativo es la verdadera ventaja: al residir las preocupaciones genéricas en módulos desacoplados y probados de forma independiente, una corrección o mejora se implementa en un solo lugar y eleva todos los productos a la vez, y cada nuevo sistema se ensambla con componentes que ya han demostrado su confiabilidad.

Qué hacemos

Desarrollo basado en AI. Construimos el sustrato para sistemas de AI de principio a fin:

- **Acceso multiproveedor a LLM** — una abstracción de primera parte sobre más de 40 proveedores (Anthropic/Claude, OpenAI, DeepSeek, Gemini, Mistral, Cohere, Groq, xAI/Grok, Qwen, Perplexity, OpenRouter, Together AI, Replicate, Cerebras, Cloudflare Workers AI, SiliconFlow y Ollama local como respaldo) detrás de una única interfaz con reintentos, disyuntores y comprobaciones de estado.
- **Orquestación de agentes** — planos de control de agentes de codificación CLI sin interfaz, flujos de trabajo agentivos basados en grafos, consenso de “debate AI” en múltiples rondas y entornos de ejecución DAG/pipeline.
- **Verificación de LLM** — una capa de confianza que evalúa modelos con una compuerta obligatoria de comprensión (“¿Ves mi código?”) más pruebas de latencia, transmisión en tiempo real, llamadas a funciones, visión y embeddings, exportando una configuración solo verificada.
- **Recuperación y memoria** — RAG, bases de datos vector, embeddings y motores de memoria fusionada para agentes (Mem0 + Cognee + Letta) con compresión de contexto ilimitado.
- **LLM defensivo** — barreras de protección, detección de PII, escenarios adversariales de equipo rojo y canonización de entradas.

La familia de productos Helix. Nuestra línea insignia abarca todo el ciclo de desarrollo de AI:

- **HelixTrack** — una alternativa de código abierto a JIRA (el buque insignia de la línea Helix-Track).

- **HelixAgent** — un servicio de LLM en conjunto que permite a múltiples modelos debatir y entregar la respuesta en la que coinciden.
- **HelixCode** — una plataforma de desarrollo distribuido de AI que divide el trabajo entre trabajadores gestionados por SSH con puntos de control y retroceso.
- **HelixLLM** — un solo binario, seis modos: inferencia compatible con OpenAI y Anthropic, desde portátiles hasta clústeres, sobre HTTP/3.
- **HelixCluster** — un sistema operativo distribuido para cómputo de AI, desde GPUs en centros de datos hasta dispositivos de borde portátiles.
- **LLMProvider / LLMOrchestrator / LLMsVerifier** — la abstracción de proveedores, el plano de control de agentes y la fuente de verdad para verificación.
- **HelixMemory, HelixSkills, HelixSpecifier, HelixBuilder, HelixTranslate, HelixTerminator, HelixGitpx, HelixOTA, HelixPlay** — memoria, habilidades gobernadas, desarrollo basado en especificaciones, construcción de aplicaciones, traducción verificada, terminales de confianza cero, Git federado, actualizaciones seguras de OTA y juegos en la nube autohospedados.

Herramientas y utilidades (vasic-digital utils). Herramientas de nivel profesional que funcionan de manera independiente:

Catalogizer (gestión de colecciones de medios multiprotocolo y cifrados), **Courses-Creator** (producción de cursos de AI a partir de Markdown a video), **VisionEngine** (visión por computadora + percepción de UI con LLM), **DocProcessor** (mapa de características a partir de documentación para control de calidad), **Docs Chain** (sincronización bidireccional de documentos y bases de datos con hash de contenido), **Herald** (notificaciones multicanal en lenguaje natural), **task_bridge** (sincronización bidireccional de tareas y tableros), y el **Vasic Digital Reusable Module Suite** — la “biblioteca estándar” `digital.vasic.*` de módulos de infraestructura, primitivas de AI y barreras de seguridad.

Automatización de infraestructura (Server Factory). Nuestro legado en DevOps: **Mail Server Factory** y el **Server Factory Core Framework**, que transforman declaraciones de JSON en servidores completamente aprovisionados y dockerizados en múltiples tipos de conexión y distribuciones de Linux, además de herramientas para imágenes de VM (Qemu-Utills, Parallels-Utills) y fábricas de servicios de soporte.

Tecnologías

Basadas en nuestra pila real:

- **Lenguajes:** Go (dominante), Kotlin y Kotlin Multiplatform, TypeScript, Python, Swift, Shell, con especificaciones formales en PL/pgSQL e incluso TLA+ para trabajo en sistemas distribuidos.
- **AI / LLM:** acceso multiproveedor (43+ adaptadores), Model Context Protocol (MCP), RAG, bases de datos vector y embeddings, algoritmos de planificación (HiPlan, MCTS, Tree of Thoughts), LLMOps, evaluación comparativa (SWE-bench/ HumanEval/MMLU) y TTS (Bark, SpeechT5).
- **Backend:** Gin (Go), gRPC + Protocol Buffers, HTTP/3 (QUIC), WebSockets, frontends Angular y React, mensajería Kafka/ RabbitMQ.
- **Datos:** PostgreSQL, SQLite, SQLCipher (cifrado en reposo), Redis, Neo4j, ClickHouse y almacenamiento de objetos (MinIO/ S3/GCS/Azure).
- **Infraestructura / DevOps:** Docker y Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/ Parallels, y CI/CD mediante GitHub Actions, Gradle y Make.
- **Pruebas / Control de calidad:** el marco anti-engaño HelixQA, arneses de desafío por módulo con puertas de mutación, go test -race, herramientas de regresión visual, pruebas en dispositivos ADB, puertas SonarQube y escaneo de seguridad (semgrep, gosec, trivy, snyk, gitleaks, nancy).

Calidad y gobernanza — nuestro elemento diferenciador

Dos pilares dan coherencia y confiabilidad a toda la flota:

- **HelixConstitution** — un manual de ingeniería universal, independiente del proyecto, distribuido como submódulo de Git e heredado por cada proyecto en una flota de más de 140 repositorios. Codifica una disciplina innegociable —barreras de evidencia contra el engaño, inmunidad a falsos positivos, seguridad de datos y hosts, normas de documentación y cobertura— que un proyecto puede ampliar, pero nunca debilitar. Una sola actualización del submódulo renueva las reglas en todas partes; los mecanismos de propagación revisan literalmente, mediante *grep*, la presencia de cláusulas obligatorias en toda la flota, y cada barrera va acompañada de una prueba de mutación que demuestra que no es un simple formalismo. La gobernanza se convierte en un hecho auditable, no en una aspiración.
- **HelixQA** — orquestación de control de calidad a prueba de engaños. Ejecuta bancos de pruebas YAML y sesiones de QA

totalmente autónomas, con LLM y visión por computadora, en Android, Android TV, Web y Desktop, y se niega a otorgar un APROBADO sin evidencia en tiempo de ejecución capturada (capturas de pantalla, *logcat*, vídeo, trazas de pila). “Lo probamos” se transforma en “aquí está el vídeo, el *logcat* y el informe”.

Declaración de posicionamiento

Cualquiera puede conectar una aplicación a un LLM. **Vasic Digital** construye lo difícil: sistemas AI verificables, reutilizables y honestos —un sustrato AI agnóstico al proveedor, un ciclo de vida de productos Helix sobre él, y una disciplina de constitución más evidencia que garantiza que lo que se entrega realmente funciona. No le pedimos que confíe en el visto bueno. Le mostramos las pruebas que lo respaldan.

Contacto

Construyamos algo verificable.

- **Correo electrónico:** i@mvasic.ru
- **GitHub:** github.com/vasic-digital